# LAPORAN PRAKTIKUM

# ALGORITMA PEMROGRAMAN 2

# MODUL 10 – PENCARIAN MIN MAX


Muhammad Habib Azzacky

109082500168

KELAS S1IF-13-06


**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

A. Dasar Teori

Pencarian nilai minimum dan maksimum merupakan salah satu operasi dasar dalam pemrosesan data yang bertujuan untuk menemukan elemen terkecil dan terbesar dari sekumpulan data yang tersimpan dalam struktur array.

Algoritma pencarian min-max bekerja dengan prinsip sequential search, yaitu menelusuri setiap elemen array satu per satu secara berurutan dari indeks pertama hingga terakhir, sambil membandingkan setiap elemen dengan nilai acuan sementara yang diinisialisasi dari elemen pertama array.

B. Guided

1. Min Max Array

```go
package main

import "fmt"

func nilaiTerbesar(array []int) (int, int) {
    var idx int
    var nilaiTerbesar int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] > nilaiTerbesar {
            nilaiTerbesar = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerbesar, idxDitemukan
}

func nilaiTerkecil(array []int) (int, int) {
    var idx int
    var nilaiTerkecil int = array[0]
    var idxDitemukan int = 0
    for idx = 1; idx < len(array); idx++ {
        if array[idx] < nilaiTerkecil {
            nilaiTerkecil = array[idx]
            idxDitemukan = idx
        }
    }
    return nilaiTerkecil, idxDitemukan
}
```

```go
func main() {
    var arrAngka [5]int = [5]int{45, 23, 98, 66, 12}

    var nilaiMaks, idxNilaiMaks int =
nilaiTerbesar(arrAngka[:])
    var nilaiMin, idxNilaiMin int =
nilaiTerkecil(arrAngka[:])

    fmt.Printf("Nilai terkecil : %d ditemukan pada index
ke-%d", nilaiMin, idxNilaiMin)
    fmt.Println()
    fmt.Printf("Nilai terbesar : %d ditemukan pada index
ke-%d", nilaiMaks, idxNilaiMaks)

}
```

Output :

```
PS D:\Kuliah\Praktikum alpro 2\109082500168_Muhammad-Habib-Azzacky> go run "d:\Kuliah\Praktik
um alpro 2\109082500168_Muhammad-Habib-Azzacky\Week 10_Modul 10\Guided\MinMax_Array.go"
Nilai terkecil : 12 ditemukan pada index ke-4
Nilai terbesar : 98 ditemukan pada index ke-2
PS D:\Kuliah\Praktikum alpro 2\109082500168_Muhammad-Habib-Azzacky>
```

2. Min Max Struct

```go
package main

import "fmt"

type mahasiswa struct {
    nama string
    nim  string
    IPK  float64
}

type arrMahasiswa [3]mahasiswa

func IPKTerbesar(array []mahasiswa) (mahasiswa, int) {
    var mhsTerbesar mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
```

```go
        if array[i].IPK > mhsTerbesar.IPK {
            mhsTerbesar = array[i]
            idxDitemukan = i
        }
    }

    return mhsTerbesar, idxDitemukan
}

func IPKTerkecil(array []mahasiswa) (mahasiswa, int) {
    var mhsTerkecil mahasiswa = array[0]
    var idxDitemukan int = 0
    var i int

    for i = 1; i < len(array); i++ {
        if array[i].IPK < mhsTerkecil.IPK {
            mhsTerkecil = array[i]
            idxDitemukan = i
        }
    }
    return mhsTerkecil, idxDitemukan
}

func main() {
    var arrMhs arrMahasiswa
    var i, j int
    for i = 0; i < len(arrMhs); i++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", i)
        fmt.Print("Masukkan nama : ")
        fmt.Scan(&arrMhs[i].nama)
        fmt.Print("Masukkan NIM :")
        fmt.Scan(&arrMhs[i].nim)
        fmt.Print("Masukkan IPK : ")
        fmt.Scan(&arrMhs[i].IPK)
        fmt.Println("=======================")
    }

    fmt.Println()
    for j = 0; j < len(arrMhs); j++ {
        fmt.Println("DATA MAHASISWA INDEKS KE-", j)
        fmt.Println("Nama : ", arrMhs[j].nama)
        fmt.Println("NIM : ", arrMhs[j].nim)
        fmt.Println("IPK : ", arrMhs[j].IPK)
        fmt.Println("=======================")
    }
```

```
    fmt.Println()

    var mhsMaks, mhsMin mahasiswa
    var idxMaks, idxMin int

    mhsMaks, idxMaks = IPKTerbesar(arrMhs[:])
    mhsMin, idxMin = IPKTerkecil(arrMhs[:])

    fmt.Println("=== DATA IPK TERKECIL & TERBESAR MAHASISWA
===")
    fmt.Printf("IPK Terbesar : %.2f, atas nama %s dengan
NIM %s, ditemukan pada indeks ke-%d", mhsMaks.IPK,
mhsMaks.nama, mhsMaks.nim, idxMaks)
    fmt.Printf("IPK Terkecil : %.2f, atas nama %s dengan
NIM %s, ditemukan pada indeks ke-%d", mhsMin.IPK,
mhsMin.nama, mhsMin.nim, idxMin)
}
```

3.

Output :

```
Masukkan NIM :wg32
Masukkan IPK : 3.22
========================

DATA MAHASISWA INDEKS KE- 0
Nama :  rijal
NIM :  wg10
IPK :  2
========================
DATA MAHASISWA INDEKS KE- 1
Nama :  zacky
NIM :  wg21
IPK :  3.45
========================
DATA MAHASISWA INDEKS KE- 2
Nama :  wafa
NIM :  wg32
IPK :  3.22
========================

=== DATA IPK TERKECIL & TERBESAR MAHASISWA ===
IPK Terbesar : 3.45, atas nama zacky dengan NIM wg21, ditemukan pada indeks ke-1IPK Terkecil
: 2.00, atas nama rijal dengan NIM wg10, ditemukan pada indeks ke-0
```

C. Unguided

1. Soal Unguided 1

1) Sebuah program digunakan untuk mendata berat anak kelinci yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat anak kelinci yang akan dijual.

**Masukan** terdiri dari sekumpulan bilangan, yang mana bilangan pertama adalah bilangan bulat N yang menyatakan banyaknya anak kelinci yang akan ditimbang beratnya. Selanjutnya N bilangan riil berikutnya adalah berat dari anak kelinci yang akan dijual.

**Keluaran** terdiri dari dua buah bilangan riil yang menyatakan berat kelinci terkecil dan terbesar.

Jawaban :

```go
package main

import "fmt"

func main() {
    var n int
    var berat [1000]float64

    fmt.Print("mau berapa kelinci? : ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        // PERBAIKAN: Gunakan koma untuk memisahkan teks dan variabel
        // Gunakan i+1 agar muncul "kelinci ke 1" bukan "kelinci ke
0"
        fmt.Print("masukan berat kelinci ke ", i+1, " : ")
        fmt.Scan(&berat[i])
    }

    // Pastikan pengecekan min/max hanya jalan jika n > 0
    if n > 0 {
        min := berat[0]
        max := berat[0]

        for i := 1; i < n; i++ {
            if berat[i] < min {
                min = berat[i]
            }
```

```
            if berat[i] > max {
                max = berat[i]
            }
        }
        fmt.Println("Berat terkecil:", min, "Berat terbesar:", max)
    } else {
        fmt.Println("Data kosong.")
    }
}
```

Output :

```
PS D:\Kuliah\Praktikum alpro 2\109082500168_Muhammad-Habib-Azzacky> go run "d:\Kuliah\Praktikum alpro 2\109082
500168_Muhammad-Habib-Azzacky\Week 10_Modul 10\Unguided\1.go"
mau berapa kelinci? : 3
masukan berat kelinci ke 1 : 10
masukan berat kelinci ke 2 : 5
masukan berat kelinci ke 3 : 3
Berat terkecil: 3 Berat terbesar: 10
```

Penjelasan :

Program ini meminta user untuk menginput jumlah kelinci yang akan ditimbang dan menentukan mana yang paling kecil dan paling besar nilainya.

2. Soal Unguided 2

Fakultas Informatika
School of Computing
Telkom University
Informatics lab

Sebuah program digunakan untuk menentukan tarif ikan yang akan dijual ke pasar. Program ini menggunakan array dengan kapasitas 1000 untuk menampung data berat ikan yang akan dijual.

**Masukan** terdiri dari dua baris, yang mana baris pertama terdiri dari dua bilangan bulat x dan y. Bilangan x menyatakan banyaknya ikan yang akan dijual, sedangkan y adalah banyaknya ikan yang akan dimasukan ke dalam wadah. Baris kedua terdiri dari sejumlah x bilangan riil yang menyatakan banyaknya ikan yang akan dijual.

**Keluaran** terdiri dari dua baris. Baris pertama adalah kumpulan bilangan riil yang menyatakan total berat ikan di setiap wadah (jumlah wadah tergantung pada nilai x dan y, urutan ikan yang dimasukan ke dalam wadah sesuai urutan pada masukan baris ke-2). Baris kedua adalah sebuah bilangan riil yang menyatakan berat rata-rata ikan di setiap wadah.

Jawaban :

```
package main

import "fmt"
```

```go
func main() {
    var x, y int
    fmt.Scan(&x, &y)

    ikan := make([]float64, x)
    for i := 0; i < x; i++ {
        fmt.Scan(&ikan[i])
    }

    jmlWadah := x / y
    if x % y != 0 {
        jmlWadah++
    }

    wadah := make([]float64, jmlWadah)
    var totalSemua float64

    for i := 0; i < x; i++ {
        indeks := i / y
        wadah[indeks] += ikan[i]
    }

    for i := 0; i < jmlWadah; i++ {
        fmt.Printf("%.2f ", wadah[i])
        totalSemua += wadah[i]
    }
    fmt.Println()

    fmt.Printf("%.2f\n", totalSemua/float64(jmlWadah))
}
```

Output :

```
PS D:\Kuliah\Praktikum alpro 2\109082500168_Muhammad-Habib-Azzacky> go run "d:\Kuliah\Praktikum alpro 2\109082
500168_Muhammad-Habib-Azzacky\Week 10_Modul 10\Unguided\2.go"
Masukkan jumlah ikan dan kapasitas wadah:
4 4
Masukkan berat masing-masing ikan:
10 4 2 5
Total berat per wadah:
21
Rata-rata berat seluruh wadah:
21
```

Penjelasan :

Program ini berfungsi untuk menghitung dan membagi total berat ikan ke dalam beberapa wadah berdasarkan kapasitas maksimal yang ditentukan, lalu menampilkan jumlah berat di setiap wadah serta nilai rata-rata berat keseluruhan wadah secara rapi dan informatif bagi pengguna.

3. Soal Unguided 3

Pos Pelayanan Terpadu (posyandu) sebagai tempat pelayanan kesehatan perlu mencatat data berat balita (dalam kg). Petugas akan memasukkan data tersebut ke dalam array. Dari data yang diperoleh akan dicari berat balita terkecil, terbesar, dan reratanya.

Buatlah program dengan spesifikasi subprogram sebagai berikut:



```
type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita; bMin, bMax *float64) {
/* I.S. Terdefinisi array dinamis arrBerat
   Proses: Menghitung berat minimum dan maksimum dalam array
   F.S. Menampilkan berat minimum dan maksimum balita */
     ...
}

function rerata (arrBerat arrBalita) real {
/* menghitung dan mengembalikan rerata berat balita dalam array */
     ...
}
```

Perhatikan sesi interaksi pada contoh berikut ini (teks bergaris bawah adalah input/read)

```
Masukan banyak data berat balita : 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.30 kg
```

Fakultas Informatika
School of Computing

Jawaban :

```go
package main

import "fmt"

type arrBalita [100]float64

func hitungMinMax(arrBerat arrBalita, n int, min, max *float64) {
```

```go
    *min = arrBerat[0]
    *max = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *min {
            *min = arrBerat[i]
        }
        if arrBerat[i] > *max {
            *max = arrBerat[i]
        }
    }
}

func rerata(arrBerat arrBalita, n int) float64 {
    var total float64 = 0
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
    return total / float64(n)
}

func main() {
    var n int
    var data arrBalita
    var beratMin, beratMax float64

    fmt.Print("Masukan banyak data berat balita: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Print("Masukan berat balita ke-", i+1, ": ")
        fmt.Scan(&data[i])
    }

    hitungMinMax(data, n, &beratMin, &beratMax)
    rata := rerata(data, n)

    fmt.Println("Berat balita minimum:", beratMin, "kg")
    fmt.Println("Berat balita maksimum:", beratMax, "kg")
    fmt.Println("Rerata berat balita:", rata, "kg")
}
```

Output :

```
PS D:\Kuliah\Praktikum alpro 2\109082500168_Muhammad-Habib-Azzacky> go run "d:\Kuliah\Praktikum alpro 2\109082
500168_Muhammad-Habib-Azzacky\Week 10_Modul 10\Unguided\3.go"
Masukan banyak data berat balita: 3
Masukan berat balita ke-1: 2
Masukan berat balita ke-2: 1
Masukan berat balita ke-3: 2.2
Berat balita minimum: 1 kg
Berat balita maksimum: 2.2 kg
Rerata berat balita: 1.7333333333333334 kg
```

Penjelasan :

Program ini dirancang untuk mengelola data berat badan balita dengan cara menerima input sejumlah data, kemudian secara otomatis mencari nilai berat badan terkecil, terbesar, dan menghitung nilai rata-ratanya untuk memberikan ringkasan statistik yang cepat dan akurat bagi pengguna.

D. Kesimpulan

Berdasarkan percobaan yang telah dilakukan, dapat disimpulkan bahwa cara terbaik untuk menemukan angka paling kecil atau paling besar adalah dengan memeriksa setiap data satu per satu secara berurutan. Proses ini melatih kita untuk menetapkan satu nilai sebagai patokan awal, lalu membandingkannya dengan sisa data lainnya hingga posisi nilai ekstrem tersebut ditemukan dengan pasti.

Selain itu, pembagian tugas dalam program menjadi bagian-bagian kecil membuat alur kerja menjadi lebih rapi dan mudah dipahami. Penggunaan cara ini tidak hanya membantu dalam mencari angka tertentu, tetapi juga mempermudah penghitungan rata-rata secara otomatis, sehingga hasil akhir yang didapat lebih akurat dibandingkan penghitungan manual.